

Computer Networks (CS-241)

Dr. Syed Muhammad Mohsin
Department of Computer Science,
University of Wah

Adapted From:

Computer Networking: A Top-Down Approach

8th edition: Jim Kurose, Keith Ross Pearson, 2020

Topics to be covered

- Application Layer
- Principles of Network Applications
- Network Application Architectures

Application layer

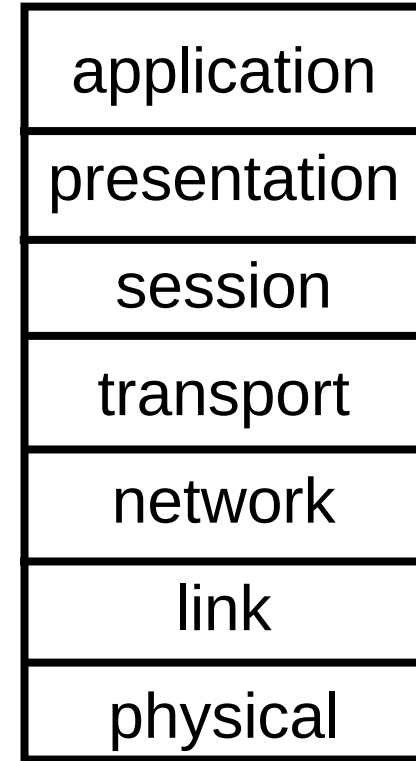
The Application Layer is the topmost layer of the OSI (Open Systems Interconnection) model. It provides services and interfaces that allow end users and applications to access the network. Key responsibilities of application layer are mentioned below.

Network services for applications: Provides protocols and services for user-level network interactions.

Data formatting and translation: Converts data from application formats into network formats (and vice versa).

Resource sharing and remote access: Allows sharing of files, printers, and other network resources.

User authentication and privacy: Supports security features such as logins, encryption, and session control.



7 layer OSI/ISO
reference model

Application layer

Feature	Description
Layer number	7 (top layer of OSI)
Purpose	Provides network services directly to user applications (software programs that people directly interact with)
Data unit	Message such as HTTP request, Email, DNS query etc.
Common protocols	HTTP/HTTPS (Web browsing), FTP (File upload/download), DNS (Translating domain names to IP addresses), SMTP (Sending emails), SNMP (Network device monitoring), Telnet/SSH (Remote access to devices), IMAP (Receiving emails)
End users	Web browsers, email clients, file transfer tools

Principles of Network Applications

The principles of network applications describe the core concepts and design guidelines used to develop applications that communicate over computer networks (like the Internet). These principles help ensure that network applications are efficient, scalable, reliable, and secure.

- **Principles of network applications**
 - Architecture Model
 - Process Communication
 - Application Protocols (E-mail, SMTP, IMAP, FTP)
 - Addressing & Naming (DNS)
 - Reliability (TCP vs UDP) etc.

Some network apps

- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video
(YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (e.g., Skype)
- real-time video conferencing
- Internet search
- remote login
- ...

Network Application Architectures

Network application architecture defines how application components (processes, clients, and servers) are organized and how they communicate over a network.

How the pieces of network applications are going to interact with each other?

- Client-server paradigm
- Peer-to-peer (P2P) architecture
- Hybrid Architecture

1. Client-server paradigm

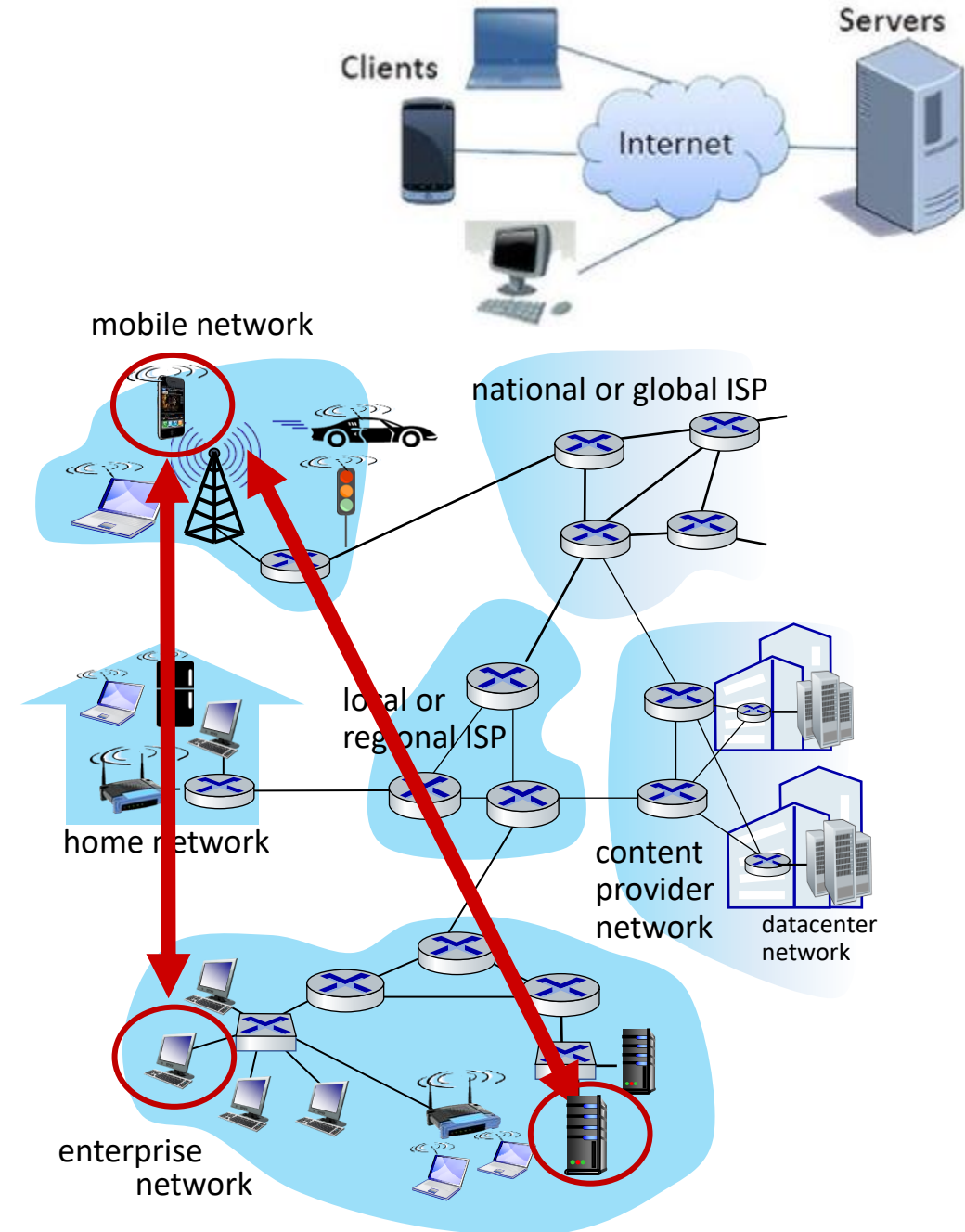
A centralized model in which a server provides services and a client requests them. For e.g., Web (HTTP: browser → web server), Email (SMTP, IMAP), Database applications.

server:

- always-on host
- permanent IP address
- often in data centers, for scaling

clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP



1. Client-server paradigm

Advantages

Centralized control

Improved security

Data backup and recovery

Efficient resource sharing

Scalability

Reliability and performance

Centralized updates

Description

All data, files, and resources are managed from a central server, improving organization and security.

Access rights and permissions are controlled by the server, reducing unauthorized access.

Centralized storage allows easy and regular data backup and recovery.

Printers, files, and applications can be shared efficiently through the server.

New clients (computers) can be added easily without major reconfiguration.

Servers are often high-performance machines that can handle multiple requests simultaneously.

Software and security updates can be applied centrally to all connected clients.

Disadvantages

High cost

Requires technical expertise

Server dependency

Maintenance overhead

Network congestion

Security risks

Description

Requires investment in dedicated server hardware and maintenance.

Setup, configuration, and management need trained network administrators.

If the server fails, clients may lose access to network resources.

Regular updates, monitoring, and backups are necessary to keep the server running efficiently.

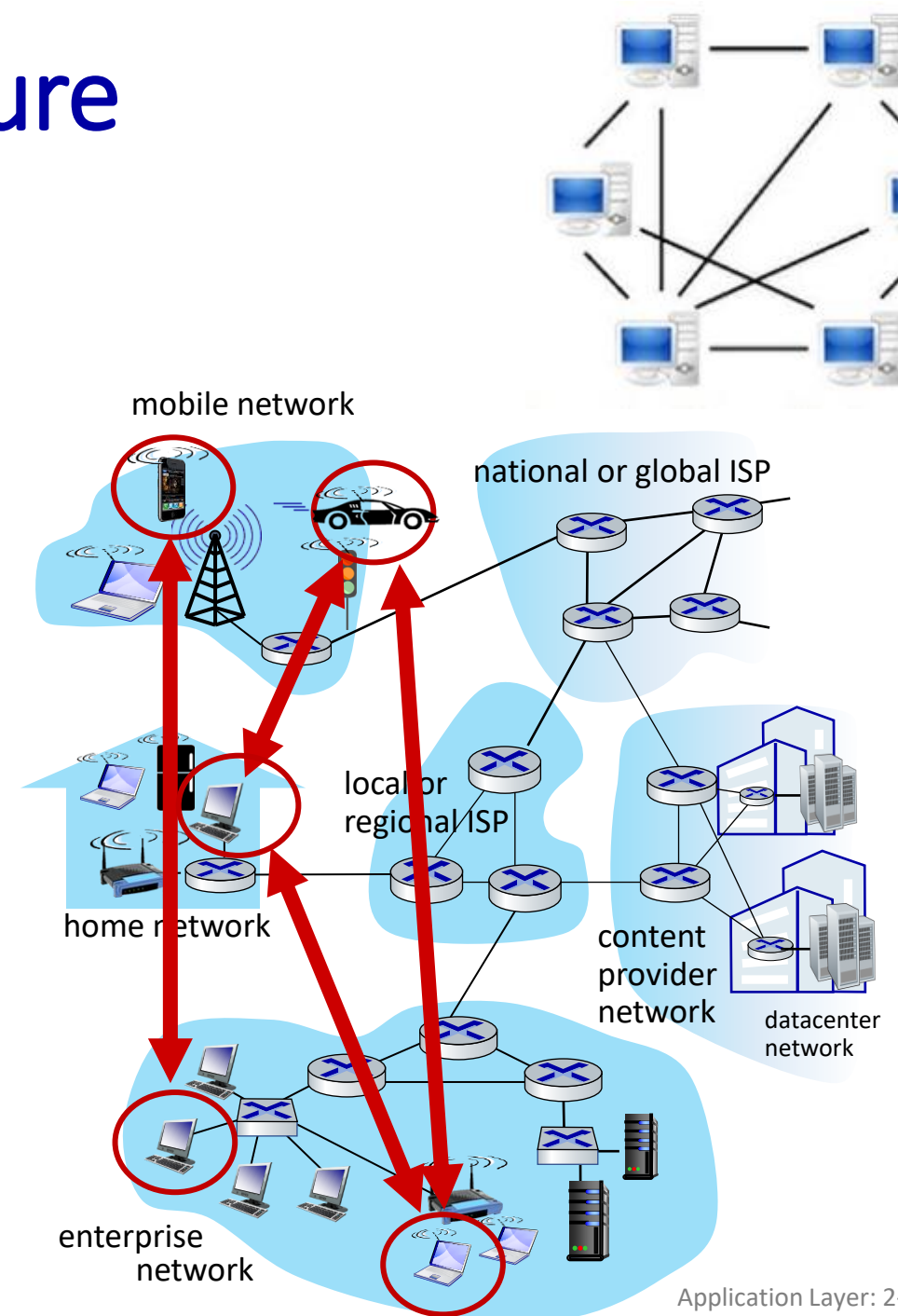
Heavy load on the server or network can slow down performance for all clients.

A compromised server can expose the entire network's data.

2. Peer-to-peer (P2P) architecture

Each peer (node) acts as both a client and a server — they share resources directly without a central server. For e.g., BitTorrent, Blockchain networks.

- Simple and less costly
- no always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers (Service provider scales at least as much as the service requested)
 - **self scalability** – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- examples: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype)



2. Peer-to-peer (P2P) architecture

Advantages

Cost-effective

Easy to set up

Resource sharing

No central point of failure

Scalable

Efficient file sharing

Description

No need for a dedicated server; reduces hardware and maintenance costs.

Simple installation and configuration; ideal for small networks.

Allows sharing of files, printers, and internet connections among peers.

Network continues functioning even if one or more nodes fail.

New nodes can be added easily without complex setup.

Multiple peers can upload/download simultaneously, improving speed (e.g., BitTorrent).

Disadvantages

Security issues

Difficult data backup

Performance degradation

No centralized management

Risk of malware and corruption

Not suitable for large networks

Description

No centralized control; difficult to enforce access permissions.

Each node maintains its own data, making centralized backup hard.

High traffic and load on individual peers can slow performance.

Troubleshooting and administration are complex.

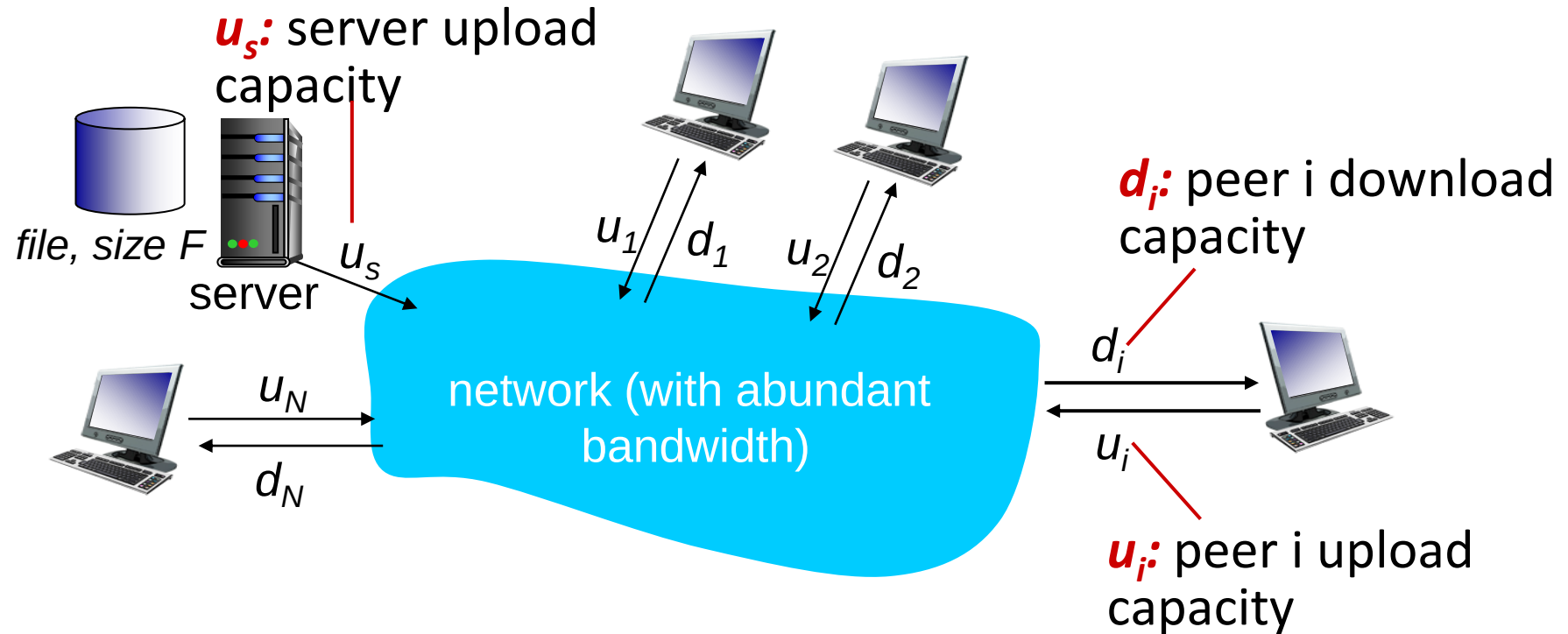
File sharing increases risk of viruses and corrupted data.

Becomes inefficient and unmanageable as network size grows.

File distribution: client-server vs P2P

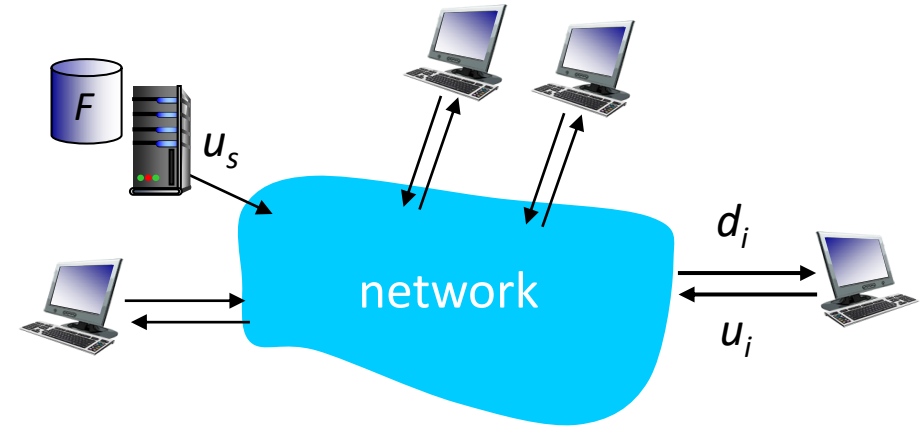
Q: how much time to distribute file (size F) from one server to N peers?

- peer upload/download capacity is limited resource



File distribution time: client-server

- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s
- **client:** each client must download file copy
 - d_{min} = min client download rate
 - min client download time: F/d_{min} (Slowest time to download)



Scalability: Time increases linearly in N

*time to distribute F
to N clients using
client-server approach*

$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

The total time to distribute file(s) = Maximum of the time taken by server to upload all copies or as long as the time taken by slowest client to download the copy

Assumption: All clients request the same file at the same time

File distribution time: P2P

- **server transmission:** must upload at least one copy:

- time to send one copy: F/u_s

- **client:** each client must download file copy

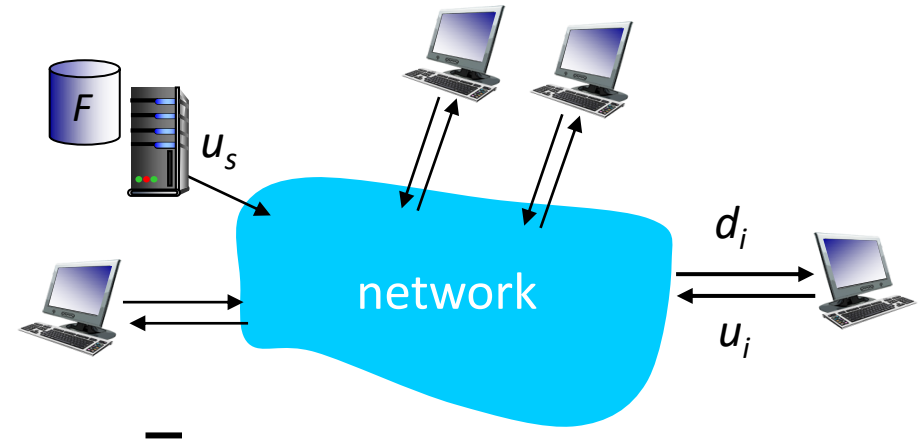
- min client download time: F/d_{min}

- **clients:** as aggregate must download NF bits

- max upload rate (limiting max download rate) is $u_s + \sum u_i$

increases linearly in N ...

... but so does this, as each peer brings service capacity



*time to distribute F
to N clients using
P2P approach*

$$D_{P2P} > \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

The file distribution can not be completed faster than the server can upload at least 1 copy or the slowest client can download one copy.

Client-server vs. P2P: example

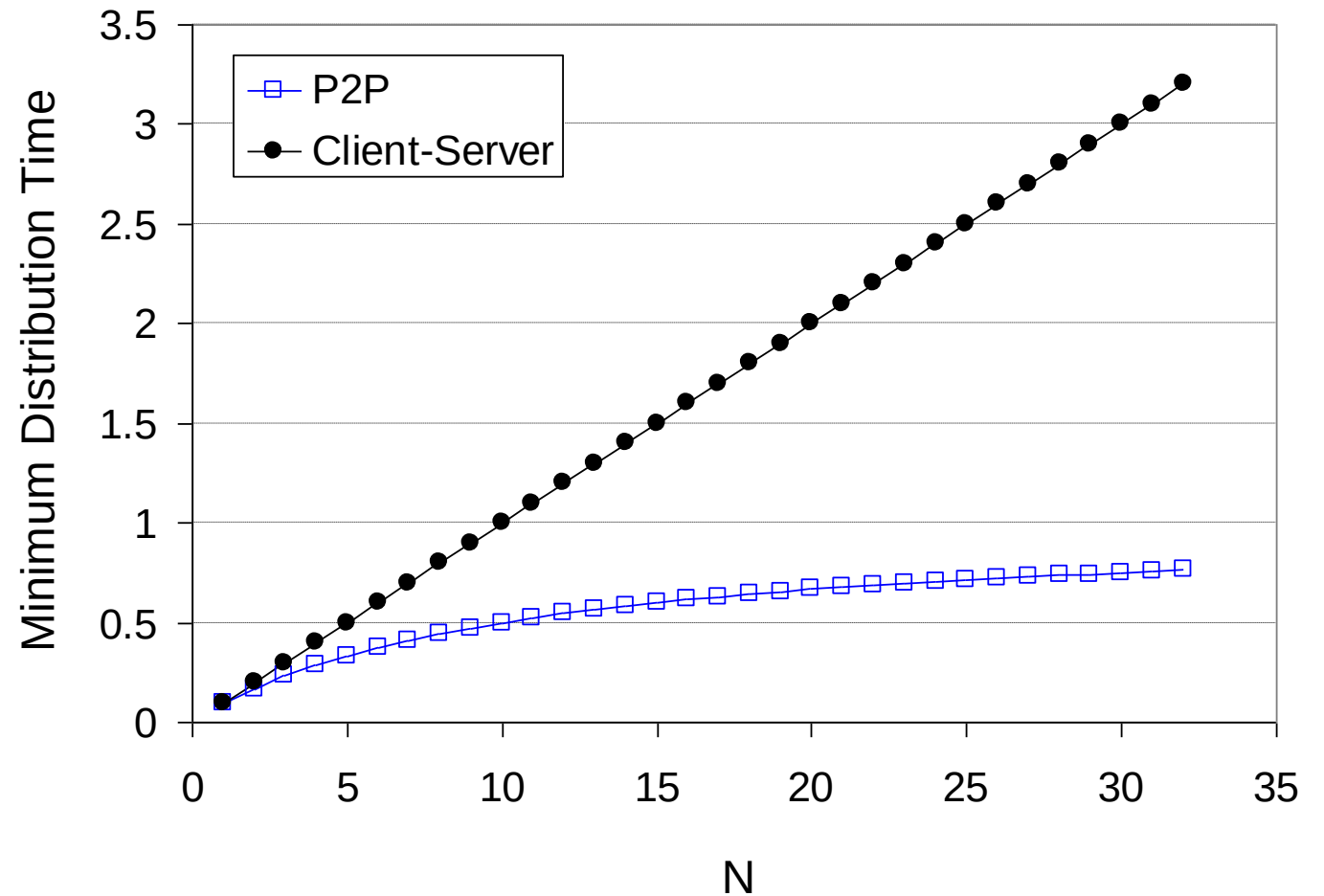
client upload rate = u (Each client can upload data to others at a rate of u units (say MB/s))

$F/u = 1$ hour (The file size F is such that if one peer uploaded the entire file alone, it would take 1 hour)

$u_s = 10u$ (The server can upload $10\times$ faster than a peer)

$d_{min} \geq u_s$ (Each client's download capacity is at least as large as the server's upload rate)

As more peers join the P2P network, the marginal increase in time is slower and performs better than a linearly increasing time (in case of client-server model)



3. Hybrid Architecture

- A combination of client–server and peer-to-peer features.
- Some functions (like indexing or authentication) are centralized, while data sharing or communication is distributed.
- For example Modern file-sharing systems (e.g., Napster), Cloud applications (central login + P2P data sharing), Online games and video conferencing apps.

3. Hybrid Architecture

Advantages

Flexibility	Combines the strengths of both client-server and P2P networks; suitable for various environments.
Efficient resource utilization	Critical resources are managed centrally, while local sharing between peers reduces server load.
Scalability	Easy to add new devices or nodes without major reconfiguration.
Improved performance	Local peer-to-peer sharing reduces network congestion and server dependency.
Enhanced reliability	If the server fails, peers can still communicate and share resources locally.
Better security control	Centralized servers manage authentication and permissions while peers handle local access.
Cost-effective	Reduces total cost compared to a fully client-server setup while retaining management benefits.

Disadvantages

Complex setup and management	Requires careful configuration and skilled administrators to balance client-server and P2P functions.
Higher initial cost	More expensive than a simple P2P network due to the inclusion of servers.
Security challenges	Managing both centralized and decentralized security policies can be difficult.
Maintenance overhead	Both server and peer systems require regular updates and monitoring.
Inconsistent performance	Peers may have variable hardware or network speeds, affecting overall efficiency.
Troubleshooting difficulty	Diagnosing problems can be harder due to mixed architecture components.